Preparação das planilhas com os dados

⬇️ Download do Boletim de Urna (BUWeb) oficial do TSE

```
# @title ⬇️ Download do Boletim de Urna (BUWeb) oficial do TSE
from pathlib import Path
import re
import zipfile

import pandas as pd
import requests

# Use UMA das opções abaixo:
# 1) informe o link completo copiado do TSE em URL_BU; ou
# 2) deixe URL_BU vazio e informe ANO, UF, TURNO e GERACAO_BU.
ANO = 2026
UF = "RN"
TURNO = 2
GERACAO_BU = "261020260742"  # pode ficar vazio se URL_BU for informado
URL_BU = ""  # exemplo: https://cdn.tse.jus.br/.../bweb_2t_RN_261020260742.zip

BASE_DIR = Path.cwd()
DADOS_DIR = BASE_DIR / "planilhas"
DOWNLOADS_DIR = DADOS_DIR / "downloads"
DADOS_DIR.mkdir(parents=True, exist_ok=True)
DOWNLOADS_DIR.mkdir(parents=True, exist_ok=True)

SEPARADOR = ";"
ENCODING_ORIGINAL = "latin-1"


def url_buweb_tse(ano, turno, uf, geracao):
    """Monta o link oficial do ZIP de Boletins de Urna do TSE."""
    return (
        f"https://cdn.tse.jus.br/estatistica/sead/eleicoes/"
        f"eleicoes{ano}/buweb/bweb_{turno}t_{uf}_{geracao}.zip"
    )


UF = UF.strip().upper()
GERACAO_BU = str(GERACAO_BU).strip()
URL_BU = str(URL_BU).strip()
if URL_BU:
    if not URL_BU.lower().startswith("https://cdn.tse.jus.br/"):
        raise ValueError("URL_BU deve apontar para o CDN oficial https://cdn.tse.jus.br/.")
    url_tse = URL_BU
    nome_zip_url = Path(url_tse.split("?", 1)[0]).name
    padrao_nome = re.fullmatch(r"bweb_([12])t_([A-Z]{2})_(\d{12})\.zip", nome_zip_url, re.I)
    padrao_ano = re.search(r"/eleicoes(\d{4})/", url_tse, re.I)
    if not padrao_nome or not padrao_ano:
        raise ValueError(
            "URL_BU deve seguir o padrão do BUWeb, por exemplo "
            "bweb_2t_RN_261020260742.zip."
        )
    TURNO = int(padrao_nome.group(1))
    UF = padrao_nome.group(2).upper()
    GERACAO_BU = padrao_nome.group(3)
    ANO = int(padrao_ano.group(1))
else:
    if not re.fullmatch(r"[A-Z]{2}", UF):
        raise ValueError("UF deve ser uma sigla de dois caracteres, por exemplo RN ou SP.")
    if TURNO not in (1, 2):
        raise ValueError("TURNO deve ser 1 ou 2.")
    if not isinstance(ANO, int) or not 2000 <= ANO <= 2100:
        raise ValueError("ANO deve ser um ano eleitoral válido, como 2026.")
    if not re.fullmatch(r"\d{12}", GERACAO_BU):
        raise ValueError(
            "Sem URL_BU, GERACAO_BU deve conter os 12 dígitos do nome exato do ZIP do TSE."
        )
    url_tse = url_buweb_tse(ANO, TURNO, UF, GERACAO_BU)

nome_zip = Path(url_tse.split("?", 1)[0]).name
if not nome_zip.lower().endswith(".zip"):
    raise ValueError("URL_BU deve apontar para um arquivo .zip do BUWeb.")
zip_path = DOWNLOADS_DIR / nome_zip
ARQUIVO_SAIDA = DADOS_DIR / f"dados_eleitorais_{UF}_{TURNO}T_{ANO}_enxuto.csv"
```

```
if not zip_path.exists():
    print(f"Baixando Boletins de Urna oficiais do TSE: {url_tse}")
    with requests.get(url_tse, stream=True, timeout=120) as resposta:
        resposta.raise_for_status()
        with zip_path.open("wb") as arquivo:
            for bloco in resposta.iter_content(chunk_size=1024 * 1024):
                if bloco:
                    arquivo.write(bloco)
else:
    print(f"ZIP já baixado: {zip_path}")

with zipfile.ZipFile(zip_path) as compactado:
    csvs = [nome for nome in compactado.namelist() if nome.lower().endswith(".csv")]
    if not csvs:
        raise RuntimeError(f"O ZIP do TSE não contém CSV: {zip_path}")
    csv_path = DADOS_DIR / Path(csvs[0]).name
    if not csv_path.exists():
        with compactado.open(csvs[0]) as origem, csv_path.open("wb") as destino:
            destino.write(origem.read())

ARQUIVO_ORIGINAL = csv_path
print(f"Fonte oficial: {url_tse}")
print(f"Arquivo extraído: {ARQUIVO_ORIGINAL}")

colunas = pd.read_csv(
    ARQUIVO_ORIGINAL,
    sep=SEPARADOR,
    encoding=ENCODING_ORIGINAL,
    dtype=str,
    keep_default_na=False,
    nrows=0,
).columns
colunas_esperadas = {
    "DS_CARGO_PERGUNTA", "DS_TIPO_VOTAVEL", "QT_VOTOS",
    "QT_APTOS", "QT_COMPARECIMENTO", "QT_ABSTENCOES",
    "QT_ELEI_BIOM_SEM_HABILITACAO",
}
faltando = sorted(colunas_esperadas.difference(colunas))
if faltando:
    raise ValueError(f"O arquivo não tem o layout de BU esperado. Colunas ausentes: {faltando}")

print("Layout de BU validado: Governador/Presidente, tipos de voto e dados eleitorais disponíveis.")
```

```
ZIP já baixado: /content/planilhas/downloads/bweb_2t_RN_261020260742.zip
Fonte oficial: https://cdn.tse.jus.br/estatistica/sead/eleicoes/eleicoes2026/buweb/bweb_2t_RN_261020260742.zip
Arquivo extraído: /content/planilhas/bweb_2t_RN_261020260742.csv
Layout de BU validado: Governador/Presidente, tipos de voto e dados eleitorais disponíveis.
```

Verificação da planilha se foi baixada corretamente

💾 Confirmação e download do CSV original

```
# @title 💾 Confirmação e download do CSV original
# O CSV original já foi extraído e salvo na célula de download.
print(f"CSV original salvo em: {ARQUIVO_ORIGINAL.resolve()}")
print(f"Tamanho do arquivo: {ARQUIVO_ORIGINAL.stat().st_size / 1024 / 1024:.2f} MB")

try:
    from google.colab import files as colab_files
except ImportError:
    colab_files = None

if colab_files is not None:
    colab_files.download(str(ARQUIVO_ORIGINAL))
else:
    print("No Jupyter local, o arquivo já está disponível no caminho acima.")
```

```
CSV original salvo em: /content/planilhas/bweb_2t_RN_261020260742.csv
Tamanho do arquivo: 26.88 MB
```

🔎 Inspeção dos dados baixados

```
# @title 🔎 Inspeção dos dados baixados
# Exibe a estrutura do CSV e amostras dos cargos trabalhados na análise.
print(f"Arquivo inspecionado: {ARQUIVO_ORIGINAL}")

# Lê apenas uma amostra para exibir o cabeçalho e os primeiros registros.
amostra = pd.read_csv(
    ARQUIVO_ORIGINAL,
```

```
    sep=SEPARADOR,
    encoding=ENCODING_ORIGINAL,
    dtype=str,
    keep_default_na=False,
    nrows=5,
)

print(f"\nNúmero de colunas: {len(amostra.columns)}")
print("Cabeçalho:")
print(list(amostra.columns))

# Conta todas as linhas sem carregar o arquivo completo na memória.
linhas_totais = 0
for bloco in pd.read_csv(
    ARQUIVO_ORIGINAL,
    sep=SEPARADOR,
    encoding=ENCODING_ORIGINAL,
    dtype=str,
    keep_default_na=False,
    usecols=[amostra.columns[0]],
    chunksize=200_000,
    low_memory=False,
):
    linhas_totais += len(bloco)

print(f"\nNúmero de linhas: {linhas_totais:,}")
print("\nDez primeiros registros do arquivo:")
display(amostra)

# O arquivo de 2026 usa DS_CARGO; versões anteriores podem usar
# DS_CARGO_PERGUNTA. Mostra uma amostra específica dos cargos analisados.
coluna_cargo = next(
    (coluna for coluna in ("DS_CARGO_PERGUNTA", "DS_CARGO") if coluna in amostra.columns),
    None,
)
if coluna_cargo:
    cargos_alvo = {"Governador", "Presidente"}
    encontrados = []
    for bloco in pd.read_csv(
        ARQUIVO_ORIGINAL,
        sep=SEPARADOR,
        encoding=ENCODING_ORIGINAL,
        dtype=str,
        keep_default_na=False,
        usecols=lambda coluna: coluna in {coluna_cargo, "NR_TURNO"},
        chunksize=200_000,
        low_memory=False,
    ):
        filtro = bloco[coluna_cargo].isin(cargos_alvo)
        if "NR_TURNO" in bloco.columns:
            filtro &= bloco["NR_TURNO"].astype(str).str.strip().eq(str(TURNO))
        encontrados.append(bloco.loc[filtro].head(5))
        if sum(len(item) for item in encontrados) >= 10:
            break

    cargos_amostra = pd.concat(encontrados, ignore_index=True).head(5) if encontrados else pd.DataFrame()
    print(f"\nAmostra de Governador/Presidente ({coluna_cargo}):")
    display(cargos_amostra)
else:
    print("\nA coluna de cargo não foi encontrada no arquivo baixado.")
```

```
Arquivo inspecionado: /content/planilhas/bweb_2t_RN_261020260742.csv

Número de colunas: 45
Cabeçalho:
['DT_GERACAO', 'HH_GERACAO', 'ANO_ELEICAO', 'CD_TIPO_ELEICAO', 'NM_TIPO_ELEICAO', 'CD_PLEITO', 'DT_PLEITO', 'NR_T

Número de linhas: 63,676

Dez primeiros registros do arquivo:
```

	DT_GERACAO	HH_GERACAO	ANO_ELEICAO	CD_TIPO_ELEICAO	NM_TIPO_ELEICAO	CD_PLEITO	DT_PLEITO	NR_TURNO	CD_ELEICA
0	26/10/2026	07:42:38	2026	0	Eleição Ordinária	3221	2026-10-25 00:00:00	2	626
1	26/10/2026	07:42:38	2026	0	Eleição Ordinária	3221	2026-10-25 00:00:00	2	626
2	26/10/2026	07:42:38	2026	0	Eleição Ordinária	3221	2026-10-25 00:00:00	2	626
3	26/10/2026	07:42:38	2026	0	Eleição Ordinária	3221	2026-10-25 00:00:00	2	626
4	26/10/2026	07:42:38	2026	0	Eleição Ordinária	3221	2026-10-25 00:00:00	2	626

5 rows × 45 columns

```
Amostra de Governador/Presidente (DS_CARGO_PERGUNTA):
```

	NR_TURNO	DS_CARGO_PERGUNTA
0	2	Governador
1	2	Governador
2	2	Governador
3	2	Governador
4	2	Governador

Próximas etapas: Gerar código com `cargos_amostra` | New interactive sheet

Clique duas vezes (ou pressione "Enter") para editar

Votação 11 · AI Agents II · MCP ·

DCA0305 · Machine Learning-Based Systems Design · UFRN

Na aula 10, as três ferramentas do estoque (buscar, detalhar e comparar) moravam dentro do notebook do agente. Escrevemos à mão a descrição de cada uma em JSON Schema e um dicionário ligando o nome à função. Hoje elas viram um **servidor MCP**, um programa separado que qualquer host consegue usar.

Parte	O que fazemos	Usa o modelo?
1 · O servidor	as três funções da aula 10 viram um `server.py` com o FastMCP	não
2 · Inspecionar	listamos e chamamos as ferramentas pela linha de comando	não
3 · JSON-RPC na mão	conversamos com o servidor escrevendo as mensagens nós mesmos	não
4 · Um cliente MCP	o mesmo, agora com o cliente Python do FastMCP	não
5 · O agente	o agente da aula 10, com as ferramentas vindas do servidor	sim, `qwen2.5:3b` no Ollama

No fim, o pedido da aula 10 (*"Quero um SUV para a família, até uns 80 mil"*) tem a mesma resposta, `c082`, `c081` e `c109`. A diferença é que agora as ferramentas chegam por um protocolo padrão.

Como rodar. No Colab, escolha o runtime **T4 GPU** (Ambiente de execução → Alterar o tipo de ambiente de execução) e rode tudo de cima a baixo. Só a Parte 5 usa o modelo.

🧰 Preparação

A célula abaixo confere cada passo e só faz o que falta. Instala o FastMCP (a biblioteca Python de MCP), baixa o estoque `cars.csv` da aula 10 e prepara o Ollama com o `qwen2.5:3b`, como na aula passada. Pode rodar quantas vezes quiser. Na primeira vez no Colab, leva uns 4 minutos. Se ela pedir para **reiniciar a sessão**, é porque o `pip` atualizou uma biblioteca que já estava carregada: reinicie e rode a célula de novo.

🧰 Preparação: confere o FastMCP, o estoque, o Ollama e o modelo, e só instala o que faltar

```
# @title 🧰 Preparação: confere o FastMCP, o estoque, o Ollama e o modelo, e só instala o que faltar
import os, shutil, subprocess, sys, time, requests

FASTMCP_VERSION = "4.1.4"
OLLAMA_URL = "http://localhost:11434"
MODEL = "qwen2.5:3b"
IN_COLAB = "google.colab" in sys.modules
sh = get_ipython().system                       # roda um comando de shell mostrando a saída na célula

os.environ["PATH"] = os.path.dirname(sys.executable) + os.pathsep + os.environ["PATH"]  # para o !fastmcp achar o

# 1 · O FastMCP está instalado e carrega por inteiro?
RESTART = ("o pip atualizou bibliotecas que esta sessão já tinha carregado. Reinicie a sessão "
           "(no Colab, Ambiente de execução → Reiniciar sessão; no Jupyter, Kernel → Restart) "
           "e rode esta célula de novo.")
try:
    import fastmcp
    print(f"✅ FastMCP {fastmcp.__version__} já instalado")
except ImportError:
    print(f"⏳ instalando o FastMCP {FASTMCP_VERSION}...")
    sh(f"pip -q install --disable-pip-version-check fastmcp=={FASTMCP_VERSION}")
    try:
        import fastmcp
    except Exception as e:
        raise RuntimeError(RESTART) from e
    print(f"✅ FastMCP {fastmcp.__version__} instalado")
try:
    from fastmcp import Client     # o cliente puxa mais bibliotecas (anyio, typing_extensions...); uma versão ve
except ImportError as e:
    raise RuntimeError(RESTART) from e

# 2 · O estoque da aula 10 está ao lado do notebook?
CSV = "/content/planilhas/bweb_2t_RN_261020260742.csv"
if os.path.exists(CSV):
    print(f"✅ {CSV} já está aqui")
else:
    url = "https://raw.githubusercontent.com/ivanovitchm/aiengineering/main/lessons/week07/cars.csv"
    r = requests.get(url)
    r.raise_for_status()
    open(CSV, "wb").write(r.content)
    print(f"✅ {CSV} baixado")

def server_is_up() -> bool:
    try:
        return requests.get(f"{OLLAMA_URL}/api/version", timeout=2).ok
    except requests.RequestException:
        return False

# 3 · O programa ollama existe nesta máquina?
if shutil.which("ollama"):
    print("✅ Ollama já instalado")
elif IN_COLAB:
    print("⏳ instalando o Ollama (cerca de 1 minuto)...")
    sh("apt-get install -y -qq zstd > /dev/null 2>&1 || "
       "(apt-get update -qq > /dev/null 2>&1 && apt-get install -y -qq zstd > /dev/null 2>&1)")
    sh("curl -fsSL https://ollama.com/install.sh | sh 2>&1 | tail -n 2")
    if not shutil.which("ollama"):
        raise RuntimeError("a instalação falhou. Rode a célula de novo.")
    print("✅ Ollama instalado")
else:
    raise RuntimeError("Ollama não encontrado. Instale em https://ollama.com/download e rode esta célula de novo.

# 4 · O servidor do Ollama está respondendo?
if server_is_up():
    print("✅ servidor do Ollama já está de pé")
else:
    print("⏳ subindo o servidor do Ollama...")
    subprocess.Popen(["ollama", "serve"], stdout=open("ollama.log", "w"), stderr=subprocess.STDOUT,
                     start_new_session=True)  # continua vivo depois que a célula termina
    for _ in range(30):
        if server_is_up():
            break
        time.sleep(1)
```

```
    if not server_is_up():
        raise RuntimeError("o servidor não respondeu. Veja o arquivo ollama.log")
    print("✅ servidor do Ollama de pé")

# 5 · O modelo já foi baixado?
installed = [m["name"] for m in requests.get(f"{OLLAMA_URL}/api/tags").json()["models"]]
if MODEL in installed:
    print(f"✅ {MODEL} já baixado")
else:
    print(f"⏳ baixando {MODEL} (~2 GB). Se a conexão cair, rode a célula de novo: o download continua.")
    sh(f"ollama pull {MODEL}")
    print(f"✅ {MODEL} baixado")

version = requests.get(f"{OLLAMA_URL}/api/version").json()["version"]
print(f"\n🚀 pronto · FastMCP {fastmcp.__version__} · Ollama {version} · modelo {MODEL} · "
      f"{'Colab' if IN_COLAB else 'máquina local'}")
```

```
✅ FastMCP 4.1.4 já instalado
✅ /content/planilhas/bweb_2t_RN_261020260742.csv já está aqui
✅ Ollama já instalado
✅ servidor do Ollama já está de pé
✅ qwen2.5:3b já baixado

🚀 pronto · FastMCP 4.1.4 · Ollama 0.40.15 · modelo qwen2.5:3b · Colab
```

🔧 Utilitários: os mesmos da aula 10 (a chamada ao modelo e duas funções para imprimir)

```
# @title 🔧 Utilitários: os mesmos da aula 10 (a chamada ao modelo e duas funções para imprimir)
import json

def llm(messages, tools=None):
    """Uma chamada ao /api/chat do Ollama. Devolve o JSON inteiro da resposta."""
    body = {"model": MODEL, "messages": messages, "stream": False,
            "options": {"temperature": 0,          # zero: a mesma entrada dá (quase) a mesma saída
                        "num_predict": 600}}       # teto de tokens por resposta, para nada travar
    if tools:
        body["tools"] = tools
    r = requests.post(f"{OLLAMA_URL}/api/chat", json=body, timeout=300)
    r.raise_for_status()
    return r.json()

def show(obj, limit=1600):
    """Imprime um dicionário como JSON legível, cortado se for longo."""
    text = json.dumps(obj, ensure_ascii=False, indent=2)
    print(text if len(text) <= limit else text[:limit] + "\n  ... (cortado)")

ICON = {"system": "⚙️ ", "user": "👤", "assistant": "🤖", "tool": "🔧"}

def show_messages(messages):
    """A conversa inteira, uma linha por mensagem."""
    for i, m in enumerate(messages):
        if m.get("tool_calls"):
            what = "pede " + " e ".join(f'{c["function"]["name"]}({json.dumps(c["function"]["arguments"], ensure_
                                        for c in m["tool_calls"])
        else:
            what = m["content"].replace("\n", " ")
        print(f"[{i}] {ICON[m['role']]} {m['role']:<9} | {what[:120]}{'…' if len(what) > 120 else ''}")

print("utilitários prontos")
```

```
utilitários prontos
```

Parte 1 · O servidor

Na aula 10, cada ferramenta tinha duas partes: a função Python e uma descrição em JSON Schema escrita à mão. Com o FastMCP, a descrição sai da própria função:

Na descrição da ferramenta	Vem de
nome	o nome da função
descrição	a docstring
parâmetros	as anotações de tipo (`float`, `str`, `list[str]`…)

A célula abaixo grava o arquivo `server.py`. O corpo das três funções é o mesmo da aula 10. As linhas novas estão marcadas com `# ←`. Além das ferramentas, o arquivo tem as outras duas primitivas do slide, um **resource** e um **prompt**.

```
%%writefile server.py
from typing import Literal
```

```
import pandas as pd
from fastmcp import FastMCP

# Ajustado para usar a planilha de votos baixada e renomeada no escopo local
bu = pd.read_csv("/content/planilhas/bweb_2t_RN_261020260742.csv", sep=";", encoding="latin-1")

mcp = FastMCP("eleicao-RN")

# Função auxiliar para conversão e validação
def _parse_int(valor, nome_campo):
    try:
        return int(float(valor)), None
    except (ValueError, TypeError):
        return None, {"error": f"parâmetro '{nome_campo}' deve ser um número válido."}

def _num(valor):
    try:
        return int(float(valor))
    except (ValueError, TypeError):
        return 0

# ─── TOOLS: o modelo decide quando chamar ───

@mcp.tool
def total_por_tipo(tipo_voto=None, cargo=None):
    """Total de votos por tipo, opcionalmente filtrado por cargo."""
    if tipo_voto is not None and tipo_voto not in ("Nulo", "Branco", "Nominal"):
        return {"error": f"tipo_voto '{tipo_voto}' inválido. Use 'Nulo', 'Branco' ou 'Nominal'."}
    if cargo is not None and cargo not in ("Governador", "Presidente"):
        return {"error": f"cargo '{cargo}' inválido. Use 'Governador' ou 'Presidente'."}

    r = bu.copy()
    r["QT_VOTOS_N"] = pd.to_numeric(r["QT_VOTOS"], errors="coerce").fillna(0)
    if tipo_voto is not None:
        r = r[r.DS_TIPO_VOTAVEL == tipo_voto]
    if cargo is not None:
        r = r[r.DS_CARGO_PERGUNTA == cargo]
    if r.empty:
        return {"error": "nenhuma seção encontrada com esses filtros"}
    return {
        "total_votos": int(r["QT_VOTOS_N"].sum()),
        "tipo_voto": tipo_voto,
        "cargo": cargo,
        "linhas": len(r),
    }

@mcp.tool
def total_sem_biometria(zona=None, municipio=None):
    """Total de eleitores sem biometria válida no arquivo inteiro.

    Para o TOTAL GERAL, chame sem argumentos: total_sem_biometria().
    """
    r = bu.copy()
    r["ZONA_N"] = pd.to_numeric(r["NR_ZONA"], errors="coerce")
    r["SECAO_N"] = pd.to_numeric(r["NR_SECAO"], errors="coerce")

    # trata string vazia como "sem filtro"
    if zona is not None and str(zona).strip() != "":
        z, err = _parse_int(zona, "zona")
        if err:
            return err
        if z == 0:
            return {"error": "zona 0 não existe. Omita o parâmetro para todas as zonas."}
        r = r[r["ZONA_N"] == z]

    if municipio is not None and str(municipio).strip() != "":
        r = r[r.NM_MUNICIPIO.str.upper() == str(municipio).upper()]

    if r.empty:
        return {"error": "nenhuma seção encontrada com esses filtros"}

    por_secao = r.drop_duplicates(subset=["ZONA_N", "SECAO_N"])
    biom = pd.to_numeric(por_secao["QT_ELEI_BIOM_SEM_HABILITACAO"], errors="coerce").fillna(0)
    aptos_s = pd.to_numeric(por_secao["QT_APTOS"], errors="coerce").fillna(0)
    total = int(biom.sum())
    aptos = int(aptos_s.sum())

    return {
        "total_sem_biometria": total,
        "total_aptos": aptos,
        "percentual": round(100 * total / aptos, 2) if aptos else None,
        "secoes": len(por_secao),
```

```
        "filtros": {"zona": zona, "municipio": municipio},
    }

@mcp.tool
def secao_details(zona, secao):
    """Todos os dados de uma seção."""
    z, err = _parse_int(zona, "zona")
    if err:
        return err
    s, err = _parse_int(secao, "secao")
    if err:
        return err

    r = bu.copy()
    r["ZONA_N"] = pd.to_numeric(r["NR_ZONA"], errors="coerce")
    r["SECAO_N"] = pd.to_numeric(r["NR_SECAO"], errors="coerce")
    r["QT_VOTOS_N"] = pd.to_numeric(r["QT_VOTOS"], errors="coerce").fillna(0)
    r = r[(r["ZONA_N"] == z) & (r["SECAO_N"] == s)]
    if r.empty:
        return {"error": f"seção {z}/{s} não encontrada"}

    primeira = r.iloc[0]
    votos = (
        r.groupby(["DS_CARGO_PERGUNTA", "DS_TIPO_VOTAVEL"], as_index=False)["QT_VOTOS_N"]
        .sum()
        .assign(QT_VOTOS=lambda d: d["QT_VOTOS_N"].astype(int))
        .drop(columns="QT_VOTOS_N")
        .to_dict(orient="records")
    )
    return {
        "zona": int(z),
        "secao": int(s),
        "municipio": primeira["NM_MUNICIPIO"],
        "aptos": int(_num(primeira["QT_APTOS"])),
        "comparecimento": int(_num(primeira["QT_COMPARECIMENTO"])),
        "abstencoes": int(_num(primeira["QT_ABSTENCOES"])),
        "sem_biometria": int(_num(primeira["QT_ELEI_BIOM_SEM_HABILITACAO"])),
        "votos": votos,
    }

@mcp.tool
def compare_secoes(zona_a, secao_a, zona_b, secao_b, tipo_voto=None):
    """Compara duas seções lado a lado, agregando por cargo."""
    if tipo_voto is not None and tipo_voto not in ("Nulo", "Branco", "Nominal"):
        return {"error": f"tipo_voto '{tipo_voto}' inválido."}

    ids = []
    for nome, valor in (("zona_a", zona_a), ("secao_a", secao_a),
                        ("zona_b", zona_b), ("secao_b", secao_b)):
        v, err = _parse_int(valor, nome)
        if err:
            return err
        ids.append(v)

    base = bu.copy()
    base["ZONA_N"] = pd.to_numeric(base["NR_ZONA"], errors="coerce")
    base["SECAO_N"] = pd.to_numeric(base["NR_SECAO"], errors="coerce")
    base["QT_VOTOS_N"] = pd.to_numeric(base["QT_VOTOS"], errors="coerce").fillna(0)
    base["AGREGADA"] = base["DS_SECOES_AGREGADAS"].astype(str).ne("#NULO#")

    secoes = []
    for i in range(0, 4, 2):
        z, s = ids[i], ids[i + 1]
        r_full = base[(base["ZONA_N"] == z) & (base["SECAO_N"] == s)]
        if r_full.empty:
            return {"error": f"seção {z}/{s} não encontrada"}
        r = r_full if tipo_voto is None else r_full[r_full.DS_TIPO_VOTAVEL == tipo_voto]
        primeira = r_full.iloc[0]
        votos_cargo = r.groupby("DS_CARGO_PERGUNTA")["QT_VOTOS_N"].sum().to_dict()
        secoes.append({
            "chave": f"{z}/{s}",
            "NR_ZONA": int(primeira["ZONA_N"]),
            "NR_SECAO": int(primeira["SECAO_N"]),
            "NM_MUNICIPIO": primeira["NM_MUNICIPIO"],
            "QT_APTOS": int(_num(primeira["QT_APTOS"])),
            "QT_COMPARECIMENTO": int(_num(primeira["QT_COMPARECIMENTO"])),
            "QT_ABSTENCOES": int(_num(primeira["QT_ABSTENCOES"])),
            "votos_por_cargo": {k: int(v) for k, v in votos_cargo.items()},
            "total_votos": int(r["QT_VOTOS_N"].sum()),
            "AGREGADA": bool(primeira["AGREGADA"]),
            "DS_SECOES_AGREGADAS": primeira["DS_SECOES_AGREGADAS"],
        })
```

```
        a, b = secoes
        best = {
            "mais_aptos": [a["chave"] if a["QT_APTOS"] >= b["QT_APTOS"] else b["chave"]],
            "mais_comparecimento": [a["chave"] if a["QT_COMPARECIMENTO"] >= b["QT_COMPARECIMENTO" ] else b["chave"]]
            "mais_abstencoes": [a["chave"] if a["QT_ABSTENCOES"] >= b["QT_ABSTENCOES"] else b["chave"]],
            "mais_votos": [a["chave"] if a["total_votos"] >= b["total_votos"] else b["chave"]],
        }
        vencedor = best["mais_votos"][0]
        resposta = (
            f"{a['chave']} tem mais votos: {a['total_votos']} vs {b['total_votos']}"
            if vencedor == a["chave"]
            else f"{b['chave']} tem mais votos: {b['total_votos']} vs {a['total_votos']}"
        )
        resultado = {
            "resposta": resposta,
            "secoes": secoes,
            "best": best,
            "tipo_voto_aplicado": tipo_voto,
        }
        agregadas = [s["chave"] for s in secoes if s["AGREGADA"]]
        if agregadas:
            resultado["aviso"] = (
                f"As seções {', '.join(agregadas)} são CONSOLIDADAS. "
                "QT_APTOS, QT_COMPARECIMENTO e QT_ABSTENCOES não são comparáveis "
                "entre elas e uma seção individual."
            )
        return resultado

    @mcp.tool
    def search_secoes(tipo_voto=None, zona=None, cargo=None, min_votos=None, top_n=5):
        """Busca seções no BU por tipo de voto, zona, cargo e mínimo de votos."""
        TIPOS = ("Nulo", "Branco", "Nominal")
        CARGOS = ("Governador", "Presidente")
        if cargo in TIPOS:
            if tipo_voto is None:
                tipo_voto = cargo
            cargo = None
        if cargo is not None and cargo not in CARGOS:
            return {"error": f"cargo '{cargo}' inválido. Use 'Governador' ou 'Presidente'."}
        if tipo_voto is not None and tipo_voto not in TIPOS:
            return {"error": f"tipo_voto '{tipo_voto}' inválido. Use 'Nulo', 'Branco' ou 'Nominal'."}

        r = bu.copy()
        r["ZONA_N"] = pd.to_numeric(r["NR_ZONA"], errors="coerce")
        r["SECAO_N"] = pd.to_numeric(r["NR_SECAO"], errors="coerce")
        r["QT_VOTOS_N"] = pd.to_numeric(r["QT_VOTOS"], errors="coerce").fillna(0)
        if tipo_voto is not None:
            r = r[r.DS_TIPO_VOTAVEL == tipo_voto]
        if zona is not None:
            zona_n, err = _parse_int(zona, "zona")
            if err:
                return err
            if zona_n == 0:
                return {"error": "zona 0 não existe. Omita o parâmetro zona para buscar em todas as zonas."}
            r = r[r["ZONA_N"] == zona_n]
        if cargo is not None:
            r = r[r.DS_CARGO_PERGUNTA == cargo]
        if min_votos is not None:
            mv, err = _parse_int(min_votos, "min_votos")
            if err:
                return err
            r = r[r["QT_VOTOS_N"] >= mv]
        if r.empty:
            return {"error": "nenhuma seção encontrada com esses filtros"}

        agg = (
            r.groupby(["ZONA_N", "SECAO_N", "NM_MUNICIPIO"], as_index=False)["QT_VOTOS_N"]
            .sum()
            .sort_values("QT_VOTOS_N", ascending=False)
        )
        top = agg.head(int(top_n or 5))
        return {
            "total_found": len(agg),
            "resumo": f"{len(agg)} seções atendem ao filtro. Abaixo, as {len(top)} com mais votos.",
            "filtros_aplicados": {"tipo_voto": tipo_voto, "zona": zona,
                                  "cargo": cargo, "min_votos": min_votos},
            "secoes": top.to_dict(orient="records"),
        }

    # ── RESOURCE: a aplicação lê, por um endereço ──
```

```
@mcp.resource("eleicao-RN://schema")
def schema() -> str:
    """As colunas do Boletim de Urna."""
    return ", ".join(bu.columns)


# ── PROMPT: o usuário escolhe usar ──

@mcp.prompt
def recomendar_analise(cargo: str, tipo_voto: str, top_n: int = 5) -> str:
    """Pede uma análise de seções com cargo e tipo de voto preenchidos."""
    return (
        f"Analise as seções com mais votos {tipo_voto} para {cargo}. "
        f"Use a ferramenta search_secoes com cargo='{cargo}', "
        f"tipo_voto='{tipo_voto}' e top_n={top_n}. "
        f"Liste as {top_n} primeiras citando zona, seção, município e o total de votos. "
        f"Depois aponte a seção com o maior valor e o quanto ela supera a segunda colocada."
    )


if __name__ == "__main__":
    mcp.run()
```

```
Overwriting server.py
```

🔍 O que aconteceu

- **A célula só gravou um arquivo.** Nada está rodando. Quem vai subir o servidor é o cliente, que o inicia como subprocesso e conversa com ele pela entrada e pela saída padrão (o transporte **stdio** do slide).
- **Sumiram a lista `TOOLS` e o dicionário `FUNCTIONS`** da aula 10, umas 40 linhas de JSON escritas à mão. O decorador `@mcp.tool` gera as duas coisas.
- **O `enum` agora é um tipo.** Na aula 10 escrevemos `"enum": ["SUV", "sedan", "hatch"]`. Aqui, `Literal["SUV", "sedan", "hatch"]` diz o mesmo para o Python e para o schema.

Parte 2 · Inspecionar, sem LLM nenhum

Na sua máquina, o **MCP Inspector** do slide abre uma interface no navegador:

```
npx @modelcontextprotocol/inspector python server.py
```

No Colab não há navegador para ele, então usamos o equivalente de linha de comando que vem com o FastMCP, `fastmcp list` e `fastmcp call`. Cada comando sobe o `server.py`, faz uma pergunta e desliga o servidor. O `2>/dev/null` no fim só esconde o log que o servidor escreve ao subir. Se um comando não mostrar nada, rode-o sem esse final para ver o erro.

🔮 **Antes de rodar.** Nunca escrevemos um JSON Schema. O que vai aparecer como entrada da `search_cars`?

```
!fastmcp list server.py --resources --prompts --input-schema 2>/dev/null
```

```
Tools (5)

  total_por_tipo(tipo_voto: any = ..., cargo: any = ...)
    Total de votos por tipo, opcionalmente filtrado por cargo.
    Input: {"type": "object", "additionalProperties": false, "properties":
{"tipo_voto": {"default": null, "title": "Tipo Voto"}, "cargo": {"default":
null, "title": "Cargo"}}}

  total_sem_biometria(zona: any = ..., municipio: any = ...)
    Total de eleitores sem biometria válida no arquivo inteiro.

Para o TOTAL GERAL, chame sem argumentos: total_sem_biometria().
    Input: {"type": "object", "additionalProperties": false, "properties":
{"zona": {"default": null, "title": "Zona"}, "municipio": {"default": null,
"title": "Municipio"}}}

  secao_details(zona: any, secao: any)
    Todos os dados de uma seção.
    Input: {"type": "object", "additionalProperties": false, "properties":
{"zona": {}, "secao": {}}, "required": ["zona", "secao"]}

  compare_secoes(zona_a: any, secao_a: any, zona_b: any, secao_b: any,
tipo_voto: any = ...)
    Compara duas seções lado a lado, agregando por cargo.
    Input: {"type": "object", "additionalProperties": false, "properties":
{"zona_a": {}, "secao_a": {}, "zona_b": {}, "secao_b": {}, "tipo_voto":
{"default": null, "title": "Tipo Voto"}}, "required": ["zona_a", "secao_a",
```

```
"zona_b", "secao_b"]}

  search_secoes(tipo_voto: any = ..., zona: any = ..., cargo: any = ...,
min_votos: any = ..., top_n: any = 5)
    Busca seções no BU por tipo de voto, zona, cargo e mínimo de votos.
    Input: {"type": "object", "additionalProperties": false, "properties":
{"tipo_voto": {"default": null, "title": "Tipo Voto"}, "zona": {"default": null,
"title": "Zona"}, "cargo": {"default": null, "title": "Cargo"}, "min_votos":
{"default": null, "title": "Min Votos"}, "top_n": {"default": 5, "title": "Top
N"}}}

Resources (1)

  eleicao-rn://schema
    schema — As colunas do Boletim de Urna.

Prompts (1)

  recomendar_analise(cargo, tipo_voto, top_n)
    Pede uma análise de seções com cargo e tipo de voto preenchidos.
```

🔍 O que aconteceu

O servidor se descreveu sozinho: Cinco ferramentas, um resource e um prompt. Em `Input` está o JSON Schema que o FastMCP gerou a partir das anotações de tipo. O `enum` do `Literal` está lá. Como `body` é opcional (`| None = None`), o schema diz "ou um desses três textos, ou `null`" (`anyOf`). A descrição de cada ferramenta é a docstring.

Agora, chamar uma ferramenta, ler o resource e pegar o prompt, como faríamos nas abas do Inspector:

```
print("🔧 TOOL total_por_tipo")
!fastmcp call server.py total_por_tipo tipo_voto=Nulo cargo=Presidente 2>/dev/null

print("\n🔧 TOOL search_secoes")
!fastmcp call server.py search_secoes tipo_voto=Nulo cargo=Presidente top_n=3 2>/dev/null

print("\n🔧 TOOL secao_details")
!fastmcp call server.py secao_details zona=3 secao=505 2>/dev/null

print("\n🔧 TOOL compare_secoes")
!fastmcp call server.py compare_secoes zona_a=3 secao_a=505 zona_b=3 secao_b=504 tipo_voto=Nominal 2>/dev/null

print("\n🔧 TOOL total_sem_biometria")
!fastmcp call server.py total_sem_biometria 2>/dev/null

print("\n📄 RESOURCE eleicao-RN://schema")
!fastmcp call server.py eleicao-rn://schema 2>/dev/null

print("\n💬 PROMPT recomendar_analise")
!fastmcp call server.py --prompt recomendar_analise cargo=Presidente tipo_voto=Nulo top_n=5 2>/dev/null
```

```
      "SECAO_N": 16,
      "NM_MUNICIPIO": "NATAL",
      "QT_VOTOS_N": 23
    },
    {
      "ZONA_N": 51,
```

```
        {
          "DS_CARGO_PERGUNTA": "Governador",
          "DS_TIPO_VOTAVEL": "Nulo",
          "QT_VOTOS": 9
        },
        {
          "DS_CARGO_PERGUNTA": "Presidente",
          "DS_TIPO_VOTAVEL": "Branco",
          "QT_VOTOS": 3
        },
        {
          "DS_CARGO_PERGUNTA": "Presidente",
          "DS_TIPO_VOTAVEL": "Nominal",
          "QT_VOTOS": 424
        },
        {
          "DS_CARGO_PERGUNTA": "Presidente",
          "DS_TIPO_VOTAVEL": "Nulo",
          "QT_VOTOS": 5
        }
      ]
    }

    🔧 TOOL compare_secoes
    {
```


🔍 O que aconteceu

Validação do servidor MCP — sem LLM

A busca devolveu **8.083 seções** com votos nulos para presidente, com **Zona 25/Seção 105 (CAICÓ)** e **Zona 4/Seção 16 (NATAL)** no topo — o mesmo gabarito da aula 10. E não houve modelo nenhum na conversa. É a regra da aula 10, "uma ferramenta se testa sem LLM, como qualquer função", agora para um servidor inteiro. É assim que a fase 2 do Trabalho 2 é validada.

Repare também na diferença entre as três primitivas. A ferramenta executa uma busca. O resource devolve um conteúdo pelo endereço `eleicao-rn://schema`. E o prompt devolve uma mensagem pronta, com os campos preenchidos, para o usuário mandar ao modelo.

O que cada primitiva devolveu

Primitiva	Chamada	Retorno
Tool	`total_por_tipo(tipo_voto="Nulo", cargo="Presidente")`	`total_votos: 61879`, `linhas: 8083`
Tool	`search_secoes(tipo_voto="Nulo", cargo="Presidente", top_n=3)`	`total_found: 8083`; topo: 25/105 CAICÓ (26), 4/16 NATAL (23), 51/176 SÃO GON
Tool	`secao_details(zona=3, secao=505)`	NATAL, 460 aptos, 432 comparecimento, 28 abstenções, 39 sem biometria; 6 linhas d
Tool	`compare_secoes(3, 505, 3, 504, "Nominal")`	`resposta: "3/505 tem mais votos: 838 vs 834"`; 504 vence em aptos, c
Tool	`total_sem_biometria()`	`total_sem_biometria: 180838`, `percentual: 6.8`, `secoes: 8115`
Resource	`eleicao-rn://schema`	45 colunas do BU, de `DT_GERACAO` a `NR_TURMA_APURADORA`
Prompt	`recomendar_analise(cargo, tipo_voto, top_n)`	Mensagem pronta com os campos preenchidos

O caso mais didático: `compare_secoes`

O `compare_secoes` mostra por que a ferramenta não é um `sum()`. A seção **3/505 vence em votos** (838 vs 834), mas a **3/504 vence em aptos** (476 vs 460), **comparecimento** (435 vs 432) e **abstenções** (41 vs 28). O campo `best` devolve o vencedor de cada critério separadamente — a ferramenta não escolhe um vencedor único, ela devolve a decisão por critério.

Atenção à diferença de totais

`total_por_tipo(tipo_voto="Nulo")` devolve **142.093** (soma dos dois cargos), enquanto `total_por_tipo(tipo_voto="Nulo", cargo="Presidente")` devolve **61.879**. O filtro de cargo corta pela metade. Confundir os dois é o erro mais fácil de cometer ao ler o retorno.

Parte 3 · JSON-RPC na mão

O `fastmcp call` escondeu as mensagens. Agora **nós** fazemos o papel do cliente. Subimos o `server.py` como subprocesso e escrevemos cada mensagem JSON-RPC na entrada dele, uma por linha. A resposta vem na saída, também uma por linha. Sem biblioteca nenhuma, só `subprocess` e `json`.

Na especificação de 2026, não há aperto de mão inicial. **Cada pedido leva no `_meta` a versão do protocolo, as capacidades do cliente e quem ele é.** O servidor não guarda nada entre um pedido e outro.

```
server = subprocess.Popen([sys.executable, "server.py"],          # o cliente sobe o servidor
                          stdin=subprocess.PIPE, stdout=subprocess.PIPE,
                          stderr=subprocess.DEVNULL, text=True)
```

```
META = {"io.modelcontextprotocol/protocolVersion": "2026-07-28",
        "io.modelcontextprotocol/clientCapabilities": {},
        "io.modelcontextprotocol/clientInfo": {"name": "aula11", "version": "1.0"}}
next_id = 1

def rpc(method, params=None, meta=True, limit=1600):
    """Envia um pedido JSON-RPC ao servidor, numa linha, e lê a resposta, na linha seguinte."""
    global next_id
    msg = {"jsonrpc": "2.0", "id": next_id, "method": method, "params": dict(params or {})}
    if meta:
        msg["params"]["_meta"] = META
    next_id += 1
    print("📤 ENVIAMOS")
    show({**msg, "params": {**msg["params"], "_meta": "{...}"}} if meta else msg)   # o _meta vai inteiro; só nã
    server.stdin.write(json.dumps(msg) + "\n")
    server.stdin.flush()
    resp = json.loads(server.stdout.readline())
    print("\n📥 RECEBEMOS")
    show(resp, limit)
    return resp

resp = rpc("server/discover")
```

```
📤 ENVIAMOS
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "server/discover",
  "params": {
    "_meta": "{...}"
  }
}

📥 RECEBEMOS
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "_meta": {
      "io.modelcontextprotocol/serverInfo": {
        "name": "eleicao-RN",
        "version": "4.1.4"
      }
    },
    "ttlMs": 0,
    "cacheScope": "private",
    "supportedVersions": [
      "2026-07-28"
    ],
    "capabilities": {
      "logging": {},
      "prompts": {
        "listChanged": false
      },
      "resources": {
        "subscribe": false,
        "listChanged": false
      },
      "tools": {
        "listChanged": false
      },
      "extensions": {
        "io.modelcontextprotocol/ui": {}
      }
    },
    "resultType": "complete"
  }
}
```

🔍 O que aconteceu

Para não poluir a tela, o `_meta` aparece como `{...}` no que enviamos, mas ele vai inteiro em todo pedido. A resposta tem o **mesmo** **`id`** do pedido. É assim que o cliente sabe a qual pedido ela responde. Em `capabilities`, o servidor diz o que oferece (`tools`, `resources` e `prompts`). Em `supportedVersions`, diz qual versão do protocolo fala.

O `server/discover` é opcional. Um cliente pode ir direto ao que interessa: listar as ferramentas.

```
resp = rpc("tools/list", limit=600)

print("\n🔎 SÓ A ENTRADA DA tipo_voto")
show(resp["result"]["tools"][0]["inputSchema"])
```

```
📤 ENVIAMOS
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/list",
  "params": {
    "_meta": "{...}"
  }
}

📥 RECEBEMOS
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "cacheScope": "private",
    "resultType": "complete",
    "tools": [
      {
        "_meta": {
          "fastmcp": {
            "tags": []
          }
        },
        "description": "Total de votos por tipo, opcionalmente filtrado por cargo.",
        "inputSchema": {
          "type": "object",
          "additionalProperties": false,
          "properties": {
            "tipo_voto": {
              "default": null,
              "title": "Tipo Voto"
            },
            "cargo": {
              "default": null,
              "t
  ... (cortado)

🔎 SÓ A ENTRADA DA tipo_voto
{
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "tipo_voto": {
      "default": null,
      "title": "Tipo Voto"
    },
    "cargo": {
      "default": null,
      "title": "Cargo"
    }
  }
}
```

🔍 O que aconteceu

É o mesmo schema que o `fastmcp list` mostrou, agora como ele viaja: um campo `inputSchema` dentro de cada ferramenta da lista. **Esse JSON é o que vai para o modelo.** Na Parte 5, é ele que entra no lugar da lista `TOOLS` da aula 10.

Agora, a chamada.

```
resp = rpc("tools/call", {"name": "total_por_tipo", "arguments": {"tipo_voto": "Nulo", "cargo": "Presidente"}})
```

```
📤 ENVIAMOS
{
  "jsonrpc": "2.0",
  "id": 3,
  "method": "tools/call",
  "params": {
    "name": "total_por_tipo",
    "arguments": {
      "tipo_voto": "Nulo",
      "cargo": "Presidente"
    },
    "_meta": "{...}"
  }
}

📥 RECEBEMOS
{
  "jsonrpc": "2.0",
  "id": 3,
  "result": {
    "content": [
      {
        "text": "{\"total_votos\":61879,\"tipo_voto\":\"Nulo\",\"cargo\":\"Presidente\",\"linhas\":8083}",
```

```
        "type": "text"
      }
    ],
    "isError": false,
    "resultType": "complete",
    "structuredContent": {
      "total_votos": 61879,
      "tipo_voto": "Nulo",
      "cargo": "Presidente",
      "linhas": 8083
    },
    "_meta": {
      "io.modelcontextprotocol/serverInfo": {
        "name": "eleicao-RN",
        "version": "4.1.4"
      }
    }
  }
}
```

🔍 O que aconteceu

O resultado vem duas vezes. Em `content`, como **texto**, que é o que vai para o modelo ler. Em `structuredContent`, como **objeto**, para um programa usar sem precisar fazer o parse do texto. E `isError: false` diz que a ferramenta rodou sem erro.

Dois tipos de erro

🔮 **Antes de rodar.** Vamos mandar dois pedidos errados: uma chamada com `total_por_tipo: "acertivo"` e um método que não existe, `tools/run`. As duas respostas vão ter a mesma forma?

```
print("═══ 1 · argumento errado ═══\n")
rpc("tools/call", {"name": "total_por_tipo", "arguments": {"tipo_voto": "Acertivo", "cargo": "Presidente"}})
print("\n═══ 2 · método que não existe ═══\n")
resp = rpc("tools/run")
```

```
═══ 1 · argumento errado ═══

📤 ENVIAMOS
{
  "jsonrpc": "2.0",
  "id": 4,
  "method": "tools/call",
  "params": {
    "name": "total_por_tipo",
    "arguments": {
      "tipo_voto": "Acertivo",
      "cargo": "Presidente"
    },
    "_meta": "{...}"
  }
}

📥 RECEBEMOS
{
  "jsonrpc": "2.0",
  "id": 4,
  "result": {
    "content": [
      {
        "text": "{\"error\":\"tipo_voto 'Acertivo' inválido. Use 'Nulo', 'Branco' ou 'Nominal'.\"}",
        "type": "text"
      }
    ],
    "isError": false,
    "resultType": "complete",
    "structuredContent": {
      "error": "tipo_voto 'Acertivo' inválido. Use 'Nulo', 'Branco' ou 'Nominal'."
    },
    "_meta": {
      "io.modelcontextprotocol/serverInfo": {
        "name": "eleicao-RN",
        "version": "4.1.4"
      }
    }
  }
}

═══ 2 · método que não existe ═══

📤 ENVIAMOS
{
  "jsonrpc": "2.0",
  "id": 5,
  "method": "tools/run",
  "params": {
```

```
      "_meta": "{...}"
    }
  }

📥 RECEBEMOS
{
  "jsonrpc": "2.0",
  "id": 5,
```

🔍 O que aconteceu

As duas formas são diferentes, como no slide dos dois tipos de erro.

	Onde aparece	Quem lê
1 · Erro de execução	dentro de `result`, com `isError: true`	**o modelo**, que pode ler a mensagem e tentar de novo com outro argume
2 · Erro de protocolo	no campo `error` do JSON-RPC, com um código (`-32601`, *Method not found*)	**o programa** do cliente, porque é bug de código

No primeiro, a mensagem do Pydantic explica o problema (*"Input should be a valid number"*). Um modelo que lê isso consegue se corrigir.

Cada pedido se sustenta sozinho

O que acontece se mandarmos um pedido **sem** o `_meta`? E se tentarmos o aperto de mão das versões antigas, o `initialize`?

```
print("═══ 3 · sem o _meta ═══\n")
rpc("tools/list", meta=False)

print("\n═══ 4 · o initialize de 2025 ═══\n")
rpc("initialize", {"protocolVersion": "2025-06-18", "capabilities": {},
                   "clientInfo": {"name": "aula11", "version": "1.0"}}, meta=False)

server.stdin.close()          # fecha a entrada: o servidor entende que acabou e termina
server.wait(timeout=10)
print("\n🛑 servidor encerrado")
```

```
═══ 3 · sem o _meta ═══

📤 ENVIAMOS
{
  "jsonrpc": "2.0",
  "id": 6,
  "method": "tools/list",
  "params": {}
}

📥 RECEBEMOS
{
  "jsonrpc": "2.0",
  "id": 6,
  "error": {
    "code": -32602,
    "message": "params._meta must be an object carrying the required 'io.modelcontextprotocol/protocolVersion' an
  }
}

═══ 4 · o initialize de 2025 ═══

📤 ENVIAMOS
{
  "jsonrpc": "2.0",
  "id": 7,
  "method": "initialize",
  "params": {
    "protocolVersion": "2025-06-18",
    "capabilities": {},
    "clientInfo": {
      "name": "aula11",
      "version": "1.0"
    }
  }
}

📥 RECEBEMOS
{
  "jsonrpc": "2.0",
  "id": 7,
  "error": {
    "code": -32022,
    "message": "connection is serving the 2026-07-28 protocol; the initialize handshake is not accepted",
    "data": {
      "supported": [
        "2026-07-28"
      ],
      "requested": "2025-06-18"
    }
```

```
    }
  }
```

```
🛑 servidor encerrado
```

🔍 O que aconteceu

Os dois foram recusados com um erro de protocolo.

- **Sem** `_meta`, o servidor não sabe qual versão o cliente fala, então não responde. Como não existe sessão, ele não tem onde ter guardado isso.
- **O** `initialize` era o aperto de mão obrigatório até 2025. Este servidor fala só a versão `2026-07-28` e responde que esse aperto de mão não é aceito. Se você encontrar um tutorial antigo começando por `initialize`, é por isso.

Foram sete pedidos, e todos tinham a mesma estrutura: `jsonrpc`, `id`, `method` e `params` na ida, e `result` ou `error` na volta. **Isso é o protocolo inteiro.** O resto é o significado de cada método.

Parte 4 · Um cliente MCP em Python

Escrever JSON-RPC na mão serve para entender o protocolo, não para trabalhar com ele. O `Client` do FastMCP faz tudo o que a Parte 3 fez: sobe o servidor, monta o `_meta`, numera os `id` e transforma as respostas em objetos Python.

O cliente é **assíncrono**, por isso o `async with` e os `await`. No notebook, eles funcionam direto na célula.

```
import sys
from pathlib import Path
from fastmcp import Client

# Para que o recurso funcione, certifique-se de que o seu server.py
# utiliza 'bu.columns' em vez de 'Votos.columns', já que 'Votos' não está definido.

# Salvamos o stderr customizado do Colab
colab_stderr = sys.stderr
# Apontamos temporariamente para o stderr nativo do sistema que possui fileno()
sys.stderr = sys.__stderr__

try:
    async with Client(Path("server.py")) as client:
        tools = await client.list_tools()
        print("🔧 ferramentas:", [t.name for t in tools])

        res = await client.call_tool("total_por_tipo", {"tipo_voto": "Nulo", "cargo": "Presidente"})
        print("\n🔧 call_tool('total_por_tipo') → res.data")
        show(res.data, limit=500)

        prompt = await client.get_prompt("recomendar_analise", {"cargo": "Presidente", "tipo_voto": "Nulo", "top_
        print("\n💬 get_prompt('recomendar_analise') →", prompt.messages[0].content.text)
finally:
    sys.stderr = colab_stderr

print("\n🛑 fora do async with, o servidor já foi encerrado")
```

```
🔧 ferramentas: ['total_por_tipo', 'total_sem_biometria', 'secao_details', 'compare_secoes', 'search_secoes']

🔧 call_tool('total_por_tipo') → res.data
{
  "total_votos": 61879,
  "tipo_voto": "Nulo",
  "cargo": "Presidente",
  "linhas": 8083
}

💬 get_prompt('recomendar_analise') → Analise as seções com mais votos Nulo para Presidente. Use a ferramenta sea

🛑 fora do async with, o servidor já foi encerrado
```

🔍 O que aconteceu

Cada `await` virou um pedido JSON-RPC igual aos da Parte 3 (o comentário de cada linha diz qual). Com isso, o servidor ficou ligado só enquanto o bloco `async with` rodou.

As três primitivas, com os três tipos de controle do slide:

Primitiva	Método do cliente	Quem decide usar
Tool	`call_tool`	o **modelo**
Resource	`read_resource`	a **aplicação**
Prompt	`get_prompt`	o **usuário**

Parte 5 · O agente da aula 10, com ferramentas MCP

O loop da aula 10 muda em **dois** lugares, e só neles:

	Aula 10	Aula 11
As descrições das ferramentas	a lista `TOOLS`, escrita à mão	`client.list_tools()`, perguntando ao servidor
A execução	`FUNCTIONS[name](**args)`, uma função no notebook	`client.call_tool(name, args)`, um pedido ao servidor

O *system prompt*, o pedido, o `max_steps` e o resto do loop continuam iguais.

5.1 · As descrições vêm do servidor

O Ollama espera cada ferramenta no formato que usamos na aula 10. A ferramenta MCP já tem as mesmas três informações. Só muda o nome das chaves.

```
import sys
from pathlib import Path
from fastmcp import Client

def to_ollama(tool):
    """Uma ferramenta MCP no formato de ferramenta do Ollama."""
    return {"type": "function",
            "function": {"name": tool.name,
                         "description": tool.description,
                         "parameters": tool.input_schema}}     # no protocolo, o campo se chama inputSchema

# Salvamos o stderr customizado do Colab
colab_stderr = sys.stderr
# Apontamos temporariamente para o stderr nativo do sistema que possui fileno()
sys.stderr = sys.__stderr__

try:
    async with Client(Path("server.py")) as client:
        TOOLS = [to_ollama(t) for t in await client.list_tools()]
finally:
    # Restauramos o stderr do Colab
    sys.stderr = colab_stderr

print(f"{len(TOOLS)} ferramentas vindas do servidor. A primeira:\n")
show(TOOLS[0])
```

```
5 ferramentas vindas do servidor. A primeira:

{
  "type": "function",
  "function": {
    "name": "total_por_tipo",
    "description": "Total de votos por tipo, opcionalmente filtrado por cargo.",
    "parameters": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "tipo_voto": {
          "default": null,
          "title": "Tipo Voto"
        },
        "cargo": {
          "default": null,
          "title": "Cargo"
        }
      }
    }
  }
}
```

Compare com a seção 3.2 da aula 10: a estrutura é a mesma (`name`, `description` e `parameters`), mas desta vez ninguém escreveu esse JSON.

5.2 · O loop, agora falando com o servidor

O `run_agent` abaixo é o da aula 10. As duas linhas que mudaram estão marcadas com `# ←`.

DEFINIÇÃO DO SYSTEM PROMPT

```
SYSTEM = f"""Agente de auditoria do BU das Eleições {ANO} da UF {UF}, turno {TURNO}.

FERRAMENTAS:
- search_secoes: busca seções e devolve o TOP N por votos. Parâmetros: tipo_voto, cargo, zona, min_votos, top_n.
- secao_details: dados completos de uma seção (aptos, comparecimento, abstenções, biometria, votos por cargo/tip
- compare_secoes: compara duas seções e identifica qual tem mais aptos, comparecimento, abstenções e votos.
- total_por_tipo: TOTAL de votos no arquivo. Parâmetros: tipo_voto, cargo.
- total_sem_biometria: total de eleitores sem biometria válida. Parâmetros: zona, municipio.

ROTEAMENTO:
- "quantos", "total", "somando", "no total" → total_por_tipo
- "quais seções", "onde", "top", "mais votos" → search_secoes
- "detalhes da seção X" → secao_details
- "compara X com Y" → compare_secoes
- "sem biometria" + "no total"/"no arquivo"/"quantos" → total_sem_biometria SEM ARGUMENTOS
- "sem biometria" + zona ou município → total_sem_biometria com o filtro citado
- NUNCA use search_secoes para responder "quantos" — ela devolve o top por seção, não o total.

PARÂMETROS:
- cargo aceita APENAS "Governador" ou "Presidente".
- tipo_voto aceita APENAS "Nulo", "Branco" ou "Nominal".
- NUNCA passe "Nulo", "Branco" ou "Nominal" em cargo.
- total_sem_biometria: para o total geral, chame SEM argumentos. NUNCA invente zona ou município. NUNCA passe st

REGRAS:
1. Se o usuário não citou um filtro, NÃO passe esse parâmetro.
"no total" ou "no arquivo" significa sem filtro.
2. Se a ferramenta devolver "error", corrija os parâmetros e chame
de novo. NÃO responda ao usuário com o erro.
Se o erro for de parâmetro inválido, leia a mensagem e ajuste:
"Nulo", "Branco" e "Nominal" são tipo_voto, nunca cargo.
Se o erro disser que um parâmetro não existe, remova-o e chame de novo.
3. Quando a ferramenta devolver dados, responda com os dados e
ENCERRE. Não acrescente frases de encerramento, ressalvas ou
avisos ao final.
4. FORA DO ESCOPO: o agente só responde sobre o BU (votos por seção,
zona, cargo e tipo). Perguntas sobre pesquisas eleitorais, nomes
de candidatos (Lula, Bolsonaro, qualquer político), geografia,
história, conhecimento geral — NÃO responda com conhecimento
próprio. Diga apenas que o dado não está no arquivo e pare.
Exemplo: "Qual é a capital do RN?" → "Esse dado não está no arquivo."
5. Em compare_secoes, use o campo "resposta" como está, citando os
DOIS números. NÃO recalcule.
6. Nunca invente números. Cite zona e seção.
7. Ao citar números, copie exatamente como aparecem na observação da ferramenta.
Nunca reformate: 4.264.428 não é 426.4428. 55.812 não é 55,812.
8. Ao listar seções, escreva "Zona X, Seção Y, município NOME, com V votos".
NUNCA escreva ZONA_N, SECAO_N, NM_MUNICIPIO ou QT_VOTOS_N na resposta.
Exemplo correto: "Zona 22, Seção 41, município CARNAÚBA DOS DANTAS, com 52 votos."
Exemplo errado: "ZONA_N:22, SECAO_N:41, NM_MUNICIPIO: CARNAÚBA DOS DANTAS"
9. Ao comparar seções, cite SEMPRE os dois totais, no formato "X tem mais: A vs B".
Nunca omita o número da seção perdedora.
10. Ao citar percentuais, copie exatamente: 6,8% — nunca 6.8%.
11. Ao citar municípios, preserve o nome exatamente como aparece na observação.
12. Ao listar seções, liste TODAS as que a ferramenta devolveu. Se o top_n foi 5, escreva 5 linhas. NUNCA diga "

Responda em português, direto."""

print("system prompt definido")
```

```
system prompt definido
```

```
PEDIDO = "Me dá os detalhes da seção 505 da zona 3."

def summary(result):
    if not isinstance(result, dict):
        return str(result)

    if "error" in result:
        return f'❌ erro: {result["error"]}'

    if "total_found" in result:
        secoes = [f'Zona {s["ZONA_N"]}, Seção {s["SECAO_N"]} ({s["NM_MUNICIPIO"]}): {s["QT_VOTOS_N"]}'
                  for s in result.get("secoes", [])]
        return f'{result["total_found"]} seções. Top: ' + " | ".join(secoes)

    if "total_sem_biometria" in result:
```

```
        pct = result.get("percentual")
        pct_txt = f" ({str(pct).replace('.', ',')}% dos aptos)" if pct is not None else ""
        total = f'{result["total_sem_biometria"]:,}'.replace(",", ".")
        return f'{total} sem biometria{pct_txt} em {result["secoes"]} seções'

    if "total_votos" in result and "secoes" not in result:
        filtro = result.get("tipo_voto") or "todos os tipos"
        cargo = result.get("cargo")
        if cargo:
            filtro = f"{filtro} para {cargo}"
        linhas = result.get("linhas")
        total = f'{result["total_votos"]:,}'.replace(",", ".")
        base = f'{total} votos ({filtro})'
        return f'{base} em {linhas} linhas' if linhas is not None else base

    if "resposta" in result:
        return result["resposta"]

    if "votos" in result:
        por_cargo = {}
        for v in result["votos"]:
            por_cargo.setdefault(v["DS_CARGO_PERGUNTA"], 0)
            por_cargo[v["DS_CARGO_PERGUNTA"]] += v["QT_VOTOS"]
        totais = " | ".join(f'{k}: {v}' for k, v in por_cargo.items())
        return (f'Seção {result["zona"]}/{result["secao"]} — {result["municipio"]}: '
                f'{result["aptos"]} aptos, {result["comparecimento"]} comparecimento. '
                f'Votos por cargo: {totais}')

    return str(result)[:300]

async def run_agent(pedido, max_steps=5):
    import sys
    messages = [{"role": "system", "content": SYSTEM},
                {"role": "user", "content": pedido}]
    print(f"👤 {pedido}\n")

    colab_stderr = sys.stderr
    sys.stderr = sys.__stderr__

    try:
        async with Client(Path("server.py")) as client:                    # ← o agente agora é um host MCP
            tools = [to_ollama(t) for t in await client.list_tools()]      # ← as descrições vêm do servidor
            for volta in range(1, max_steps + 1):
                print(f"── volta {volta} " + "─" * 60)
                msg = llm(messages, tools=tools)["message"]                 # REASON: o modelo decide
                messages.append(msg)
                if not msg.get("tool_calls"):                               # texto em vez de ferramenta: fim do l
                    print("🤖 o modelo respondeu com texto → fim do loop\n")
                    print(msg["content"])
                    return messages
                for c in msg["tool_calls"]:
                    name, args = c["function"]["name"], c["function"]["arguments"]
                    print(f"🤖 o modelo pediu   → {name}({json.dumps(args, ensure_ascii=False)})")
                    res = await client.call_tool(name, args, raise_on_error=False)   # ← ACT: o servidor executa
                    if res.is_error:
                        print(f"📡 o servidor       → isError: {res.content[0].text[:80]}")
                    else:
                        print(f"📡 o servidor       → {summary(res.data)}")
                    messages.append({"role": "tool", "tool_name": name,             # OBSERVE: o resultado volta
                                     "content": res.content[0].text})
    finally:
        sys.stderr = colab_stderr

    print(f"(parei depois de {max_steps} voltas sem uma resposta final)")
    return messages

conversa = await run_agent(PEDIDO)
```

```
👤 Me dá os detalhes da seção 505 da zona 3.

── volta 1 ────────────────────────────────────────────────────────────
🤖 o modelo pediu   → secao_details({"secao": "505", "zona": "3"})
📡 o servidor       → Seção 3/505 — NATAL: 460 aptos, 432 comparecimento. Votos por cargo: Governador: 432 | Pres
── volta 2 ────────────────────────────────────────────────────────────
🤖 o modelo respondeu com texto → fim do loop

Zona 3, Seção 505, município NATAL, com 460 aptos, 432 comparecimento, 28 abstenções, 39 sem biometria válida, te
```

🔍 O que aconteceu

O agente como host MCP

Até aqui o agente chamava funções Python direto. Agora ele é um **host MCP**: sobe o `server.py` como subprocesso, descobre as ferramentas pelo `tools/list` e executa cada uma pelo `tools/call`. O modelo continua decidindo o que chamar — mas quem executa é o servidor, não o notebook.

O que mudou em relação ao loop anterior

Antes	Agora
`FUNCTIONS[name](**args)` — chamada direta em Python	`await client.call_tool(name, args)` — chamada via MCP
`TOOLS` escrito à mão no notebook	`[to_ollama(t) for t in await client.list_tools()]` — vem do servidor
`summary(result)` sobre o dicionário	`summary(res.data)` sobre o retorno do MCP
Erro capturado com `try/except`	`raise_on_error=False` + `res.is_error`

O ponto central: **as descrições das ferramentas não estão mais no notebook**. Elas vêm do `server.py` pelo `tools/list`. Se você mudar a docstring de uma tool no servidor, o modelo vê a mudança sem tocar no agente.

As quatro fases do loop

O loop segue o padrão ReAct, agora com o servidor no meio:

- **REASON** — `llm(messages, tools=tools)` devolve a decisão do modelo
- **ACT** — `client.call_tool(name, args)` executa no servidor
- **OBSERVE** — `res.content[0].text` volta como mensagem `tool`
- **Fim** — quando o modelo responde texto em vez de pedir ferramenta

O tratamento de erro

```
res = await client.call_tool(name, args, raise_on_error=False)
if res.is_error:
    print(f"📡 o servidor       → isError: {res.content[0].text[:80]}")
else:
    print(f"📡 o servidor       → {summary(res.data)}")
```

```
    conversa = await run_agent("Compara os votos nominais da seção 505 com a 504 da zona 3. Qual tem mais?")
    print("\n" + "=" * 72 + "\n")
    conversa = await run_agent("Quais seções têm mais votos nulos para governador?")
    print("\n" + "=" * 72 + "\n")
    conversa = await run_agent("Quantos eleitores estão sem biometria válida no total?")
    print("\n" + "=" * 72 + "\n")
    conversa = await run_agent("Compara os votos nulos da seção 41 da zona 22 com a 35 da zona 24.")
    print("\n" + "=" * 72 + "\n")
    conversa = await run_agent("Quais seções têm mais votos nulos para governador?")

    print("\n📜 A CONVERSA COMPLETA")
    show_messages(conversa)
```

```
    Zona 24, Seção 35, município PARELHAS, com 37 votos.
    Zona 22, Seção 41, município CARNAÚBA DOS DANTAS, com 36 votos.
    Zona 69, Seção 106, município NATAL, com 35 votos.
    Zona 22, Seção 26, município CARNAÚBA DOS DANTAS, com 29 votos.

    Essas seções representam as cinco com mais votos nulos para o cargo de Governador, conforme os dados encontrados

    ═══════════════════════════════════════════════════════════════════════
```

```
═══════════════════════════════════════════════════════════════════════

👤 Quais seções têm mais votos nulos para governador?

── volta 1 ──────────────────────────────────────────────────────────────
🤖 o modelo pediu   → search_secoes({"cargo": "Governador", "tipo_voto": "Nulo", "top_n": 5})
📡 o servidor       → 8095 seções. Top: 24/35 (PARELHAS): 37 | 22/41 (CARNAÚBA DOS DANTAS): 36 | 69/106 (NATAL)
── volta 2 ──────────────────────────────────────────────────────────────
🤖 o modelo respondeu com texto → fim do loop

As seções com mais votos nulos para o cargo de Governador são:

Zona 24, Seção 35, município PARELHAS, com 37 votos.
Zona 22, Seção 41, município CARNAÚBA DOS DANTAS, com 36 votos.
Zona 69, Seção 106, município NATAL, com 35 votos.
Zona 22, Seção 26, município CARNAÚBA DOS DANTAS, com 29 votos.

Essas seções representam as cinco com mais votos nulos para o cargo de Governador, conforme os dados encontrados

📜 A CONVERSA COMPLETA
[0] ⚙️  system    | Agente de auditoria do BU das Eleições 2026 da UF RN, turno 2.  FERRAMENTAS: - search_secoes
[1] 👤 user      | Quais seções têm mais votos nulos para governador?
[2] 🤖 assistant | pede search_secoes({"cargo": "Governador", "tipo_voto": "Nulo", "top_n": 5})
[3] 🔧 tool      | {"total_found":8095,"resumo":"8095 seções atendem ao filtro. Abaixo, as 5 com mais votos.","
[4] 🤖 assistant | As seções com mais votos nulos para o cargo de Governador são:  Zona 24, Seção 35, município
```

🔍 O que aconteceu

Na primeira pergunta, o modelo escolheu `car_details("c109")` lendo só as descrições que vieram do servidor e respondeu que o Nivus está em Natal e é vermelho.

Na segunda, foram três voltas. Primeiro `search_cars`, depois `compare_cars(["c081", "c082"])` com os ids que o próprio modelo tirou da busca, e então a resposta. É o mesmo ReAct da aula 10, e cada ação passou pelo servidor.

Os números continuam vindo das ferramentas e o texto continua sendo do modelo. Durante a preparação, ele escolheu o `c081` dizendo que ele tem "um menor número de quilômetros rodados" e "é o mais novo". O trace mostra outra coisa: em `lowest_km`, quem venceu foi o `c082`, e em `newest` deu empate (`c081/c082`). A escolha em si se defende, porque o `c081` venceu sem empate em `cheapest` e `most_economical`. A justificativa, não. Trocar o transporte das ferramentas não muda essa lição. **O texto se confere.**

Resumo

```
👤 pedido
   │
   ▼
🤖 agente (host + cliente MCP) ── tools/call · JSON-RPC · stdio ──►  📡 server.py (FastMCP)
   ▲                                                                      │
   └──────────────── resultado: content, structuredContent, isError ◄─────┘
   │
   ▼  (quando o modelo responde com texto)
💬 resposta
```

1. **Um servidor MCP é um programa separado** que oferece ferramentas, resources e prompts. Com o FastMCP, são funções Python com um decorador, e a descrição sai da docstring e das anotações de tipo.
2. **O protocolo são mensagens JSON-RPC.** `jsonrpc`, `id`, `method` e `params` na ida, `result` ou `error` na volta. Na versão de 2026, cada pedido leva o próprio `_meta` e não há sessão.
3. **Há dois tipos de erro.** O erro de execução (`isError`) é para o modelo ler. O erro de protocolo (`error`) é para o programa.
4. **O agente quase não muda.** As descrições vêm de `list_tools` e a execução vai por `call_tool`. O loop, o *system prompt* e a resposta são os mesmos da aula 10.

Daqui para a fase 2 do Trabalho 2

Pegue as três ferramentas da fase 1 do seu grupo (`search`, `details` e `compare`) e transforme-as num servidor MCP, como a Parte 1. Valide o servidor no **MCP Inspector** na sua máquina, listando e chamando as três ferramentas sem LLM, como a Parte 2. Depois troque, no seu agente, a lista de ferramentas e a execução pelas chamadas ao servidor, como a Parte 5. **Prazo: 09/11, no início da aula.**